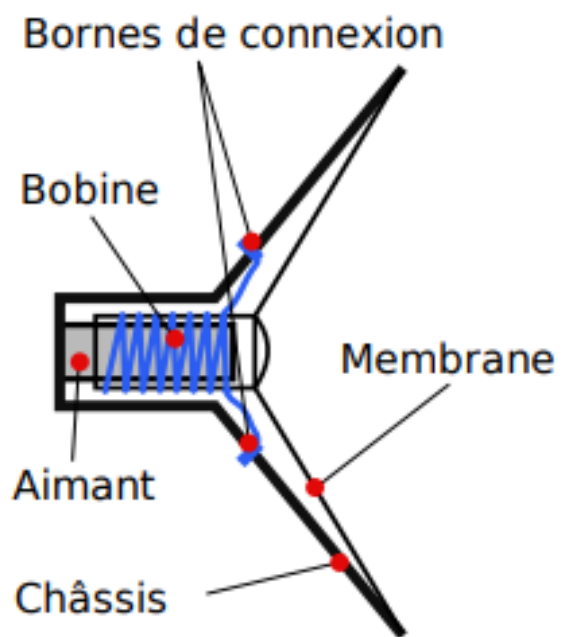
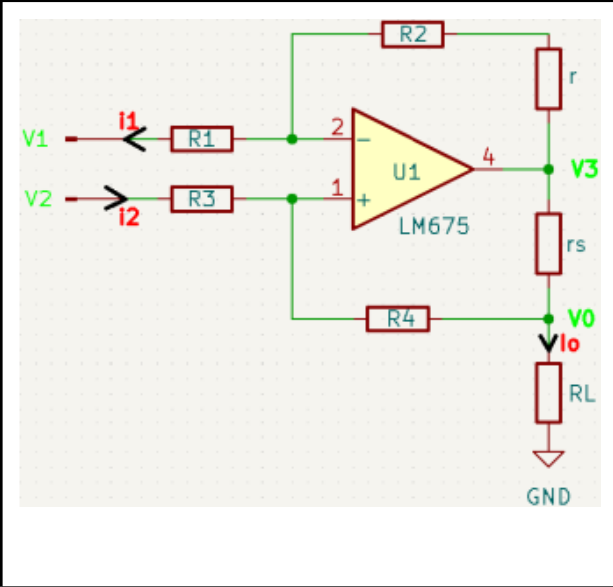


<u>Introduction du projet</u>	<u>But du projet</u>	<u>Compétences Mises en Œuvre</u>
L'analyse précise de l'impédance des haut-parleurs est essentielle pour garantir la qualité sonore et la fiabilité des systèmes audio. Ce projet vise à concevoir et à mettre en œuvre un banc de test complet pour mesurer l'impédance des haut-parleurs d'abord de manière analogique puis de manière numérique . L'objectif principal est la conformité des haut-parleurs aux spécifications techniques.	Le but du projet est de créer une chaîne d'acquisition capable de mesurer avec précision l'impédance des haut-parleurs. Cela inclut la conception d'un banc de test, la simulation théorique à l'aide de MATLAB, et l'utilisation de techniques de filtrage pour isoler les signaux pertinents. Le projet vise également à automatiser le processus de mesure afin de faciliter les tests en milieu industriel ou lors d'événements tels que des concerts ou des spectacles.	- Électronique analogique : Compréhension des circuits analogiques, notamment les amplificateurs de courant, les filtres et les multiplicateurs analogiques. - Traitement du signal : Utilisation de la corrélation sinusoïdale et de la transformation de Fourier rapide (FFT) pour analyser les signaux en fréquence. - Programmation : Développement de scripts MATLAB pour la simulation et l'analyse théorique, ainsi que de programmes Arduino pour l'acquisition des données. - Instrumentation et mesure : Manipulation d'oscilloscopes, de générateurs de fonctions, et d'autres appareils pour effectuer des mesures précises.

	<u>Composition d'un Haut-Parleur</u>
	Un haut-parleur est un transducteur électromécanique qui convertit un signal électrique en onde sonore. Ses principaux composants sont : - La bobine : Parcourue par le courant audio, elle se déplace dans un champ magnétique, entraînant la membrane. - L'aimant : Génère le champ magnétique dans lequel se déplace la bobine. - La membrane : Transforme les mouvements de la bobine en ondes sonores. - Le châssis : Structure mécanique qui soutient l'ensemble des composants.

	<u>Méthodes de Filtrage Utilisées</u>	
<i>Analogique :</i>		<i>Numérique :</i>
- Corrélation Sinusoïdale : Permet d'extraire la composante fondamentale du signal en éliminant les harmoniques et le bruit.	Cette méthode de filtrage est commune à l'analogique et au numérique. En analogique nous allons utiliser des multiplieurs MPY634KP pour faciliter l'extraction des parties réelles et imaginaires du signal. Et numériquement cela est encore plus simple car il suffit d'acquérir les signaux puis de les multiplier entre-eux	- Corrélation Sinusoïdale : Permet d'extraire la composante fondamentale du signal en éliminant les harmoniques et le bruit.
- Filtre de Rauch : Un filtre passe-bas analogique utilisé pour atténuer les hautes fréquences indésirables et isoler la fréquence fondamentale	En analogique nous allons utiliser un filtre passe-bas du second ordre qui va nous permettre d'éliminer les composantes intéressantes pour notre étude Numériquement ce sera la combinaison de la corrélation sinusoïdale et de la FFT qui va nous permettre de garder seulement ce qui nous intéresse , à savoir un signal sans bruit et harmoniques	FFT : La FFT est appliquée aux signaux numérisés pour obtenir une représentation fréquentielle et identifier les images d'impédance.

Lundi Matin : Etude source de courant de Howland

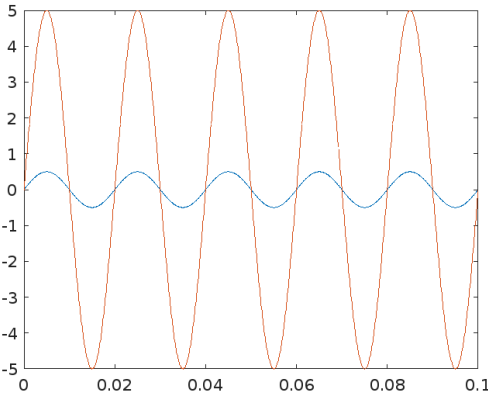
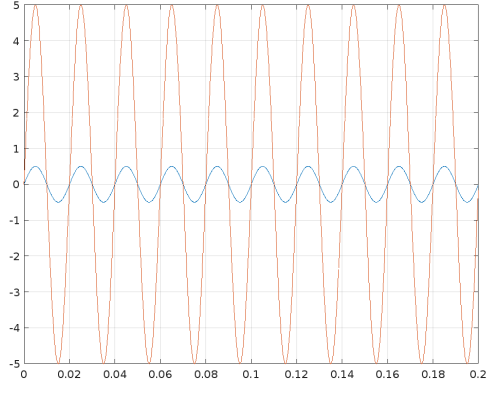


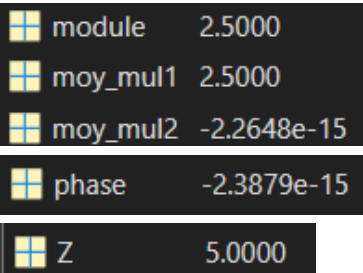
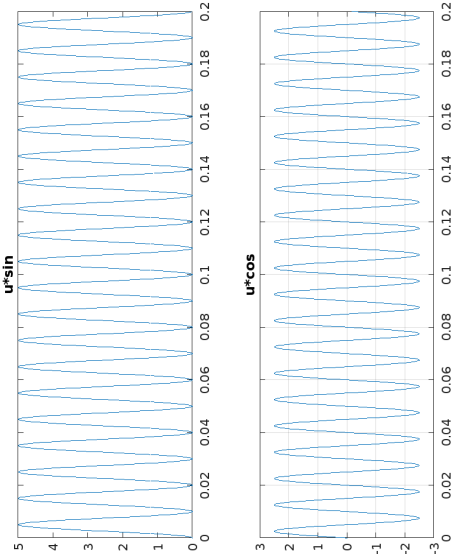
Ce circuit agit comme un générateur de courant et nous permet de générer un courant constant pour notre haut parleur.

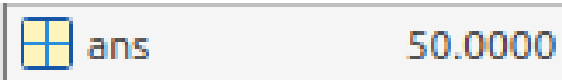
- Nous avons dû étudier son fonctionnement et en déduire la formule du courant I_0 qu'il génère
- Finalement nous avons trouvé que I_0 est égal à $-2,5 \cdot V_{in}$, V_{in} étant la tension qui entre dans le howland

Lundi Après-midi : Etude Matlab

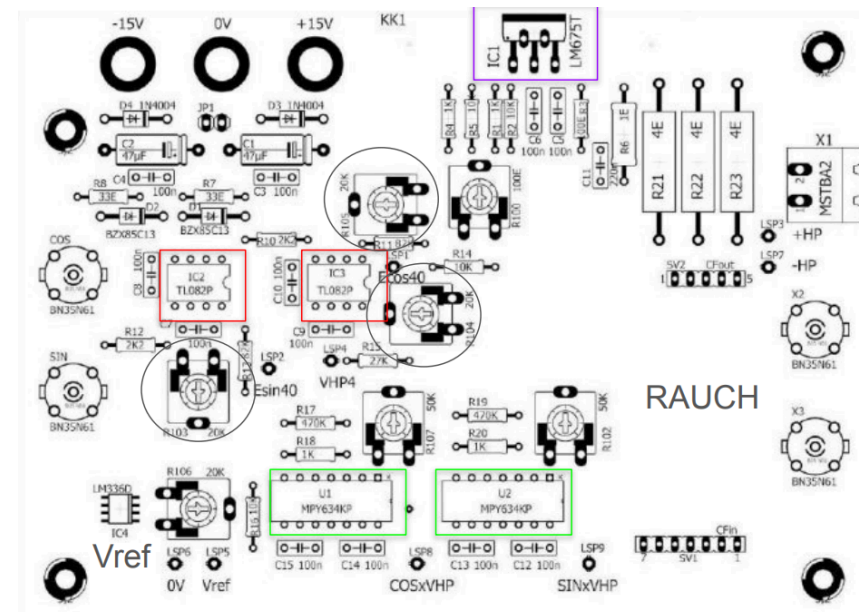
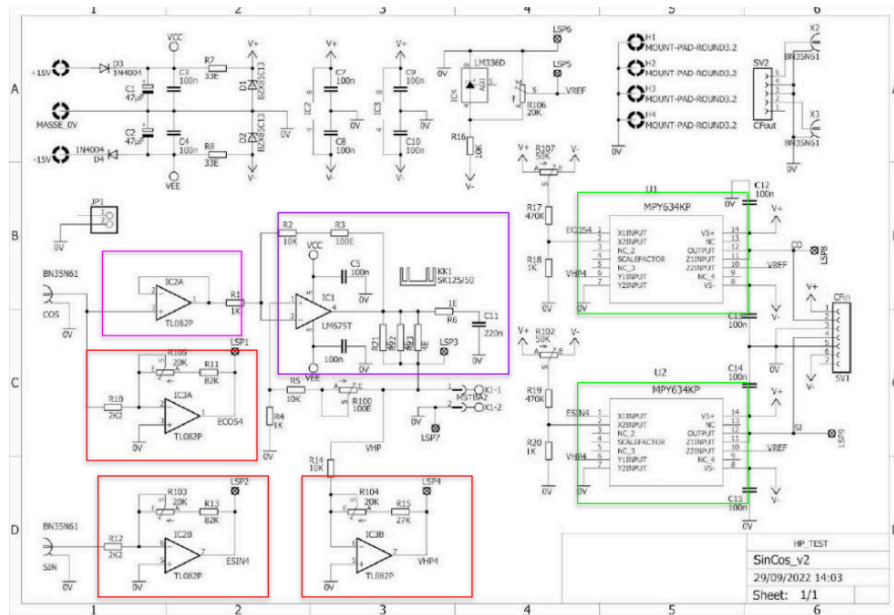
	Script	Graphique généré	Explications , problèmes rencontrés , solutions
--	--------	------------------	---

Question 1 On considère que le courant dans la résistance est de la forme $i = I \cdot \sin(2 \cdot \pi \cdot f \cdot t)$ avec $I = 0,5A$ et $f = 50Hz$. Si $R = 10 \Omega$. Définir U et de l'équation de la tension instantanée $u = U \cdot \sin(2 \cdot \pi \cdot f \cdot t + \varphi)$ aux bornes de la résistance.	<pre> I = 0.5; f = 50; R = 10; U = R*I; phi = 0; t=0:0.0001:0.1; i = I*sin(2*pi*f*t+0); u = U*sin(2*pi*f*t+phi); plot(t,i); hold on; plot(t,u); </pre>		Dans cette question nous avons pu tracer le courant et la tension en fonction de données qui nous étaient données dans l'énoncé - On trouve que phi est égal à 0 car dans une résistance le courant est en phase avec la tension - U est égal à $R \cdot I$ selon la loi d'ohm donc $10 \cdot 0.5$ soit 5V
Question 2 et 3 2. Pour générer le courant i dans Matlab on va créer un signal échantillonné à la fréquence $f_e = 10 \text{ kHz}$. Il faut que l'on crée le vecteur temps t, définir ce vecteur en partant de $t = 0$ et comme on a une fréquence $f=50Hz$ on prendra 10 périodes du signal moins un échantillon quelle que soit la fréquence f. Pour pouvoir intégrer facilement sous Matlab, nous utiliserons un nombre entier de périodes. Réaliser un script Matlab dans votre répertoire courant. 3. Maintenant que nous avons le vecteur temps, créez les vecteurs i et u représentant les valeurs instantanées. Faire un plot de ces deux vecteurs	<pre> clear all; close all; I = 0.5; f = 50; T = 1/f; fe = 10000; Te = 1/fe; R = 10; U = R*I; phi = 0; nbre_periodes = 10; temps_total = nbre_periodes*T; t=0:Te:(temps_total-Te); i = I*sin(2*pi*f*t); u = U*sin(2*pi*f*t+phi); plot(t,i); hold on; plot(t,u); grid on; </pre>		Dans cette question nous avons voulons créer un signal échantillonné avec 10 périodes moins un échantillon et cela quelle que soit la fréquence f - f_e est égal à 10 kHz donc T_e est égal à $1/10000$ - le vecteur temps commence à 0 avec un pas de T_e la période d'échantillonnage et s'arrête au bout de 10 périodes moins T_e qui correspond à un échantillon - à partir de là nous pouvons créer les vecteurs i et u qui correspondent aux valeurs instantanées du courant et de la tension pour les afficher sur notre graphique

4. Créez les deux signaux multiplicateurs ($\sin(\omega t)$ et $\cos(\omega t)$). Le signal $\sin(\omega t)$ est en phase avec le signal $A.\sin(\omega t)$ de l'amplificateur en courant. 5. Réalisez les multiplications avec la tension u aux bornes de la résistance pure 6. Créez une nouvelle figure et faire un plot du résultat des multiplications. Conclure sur la valeur moyenne de chaque signal. 7. Pour trouver les valeurs de la partie réelle et imaginaire du fondamental, on doit intégrer le résultat des multiplications. Pour cela nous utiliserons l'instruction de la moyenne (mean). 8. Calculer le module et la phase, conclure sur les résultats.	<pre>clear all; close all; I = 0.5; f = 50; T = 1/f; fe = 10000; Te = 1/fe; R = 10; U = R*I; U1=2; U2=2; nbre_periodes = 10; temps_total = nbre_periodes*T; t=0:Te:(temps_total-Te); u = U*sin(2*pi*f*t); a = sin(2*pi*f*t); b = sin(2*pi*f*t+pi/2); mul1 = a.*u; mul2 = b.*u; moy_mul1=mean(mul1); moy_mul2=mean(mul2); module = sqrt(moy_mul1^2 + moy_mul2^2); phase = atan(mul1/mul2); Z = module/I; subplot(2,1,1); plot(t,mul1); title('u*sin'); hold on; subplot(2,1,2); plot(t,mul2); title('u*cos'); grid on;</pre>	 	Dans cette question nous avons d'abord généré deux signaux : un cosinus et un sinus. Pour ensuite les multiplier avec notre tension u et calculer leur moyenne. Pour ensuite calculer le module et la phase. - Graphiquement nous avons pu voir que la valeur moyenne du sinus était de 2.5V et celle du cosinus de 0V. - Ce qu'on a pu vérifier avec la fonction mean qui calcule la moyenne des multiplications (intégrale) Lorsque on a calculé le module et la phase pour ensuite calculer Z nous avons remarqué que l'impédance de sortie est 2 fois plus petite que l'impédance d'entrée, c'est pourquoi nous allons passer à une étude mathématique pour comprendre d'où vient ce problème
---	---	--	---

Mardi Matin : suite Etude Matlab			
9. Apportez cette modification à votre script Matlab et vérifiez maintenant que les valeurs sont bonnes. 10. Nous allons maintenant vérifier notre script Matlab pour tous types d'impédance. Pour cela nous allons transformer le script en fonction et lui passer les paramètres f, A, Z, . 10. Réalisez la fonction correlationSin(50, 0.5, 10, 0) ; Mettre les instructions d'affichage (figure, grid, plot...) en commentaire.	<pre>function [z1,phase1]=correlationSin(f,l, Z,phi) T = 1/f; fe = 10000; Te = 1/fe; U = Z*I; U1=2; U2=2; nbre_periodes = 10; temps_total = nbre_periodes*T; t=0:Te:(temps_total-Te); u = U*sin(2*pi*f*t+phi); a = U1*sin(2*pi*f*t); b = U2*sin(2*pi*f*t+pi/2); mul1 = a.*u; mul2 = b.*u; moy_mul1=mean(mul1); moy_mul2=mean(mul2); module = sqrt(moy_mul1^2 + moy_mul2^2); phase1 = atan(mul1/mul2); z1 = module/l; end correlationSin(50,0.5,50,pi/2);</pre>		$B1 = \frac{U*U1}{2} \sqrt{\cos^2(\phi) + \sin^2(\phi)}$ D'où $B1 = \frac{U*U1}{2}$ Si $B1 = U$ alors $U = \frac{U*U1}{2}$ Pour obtenir $B1 = U$ il faut que $U1 = U2 = 2$ Maintenant en apportant la modification à notre script MatLab nous remarquons que le résultat est tout de suite plus correct et cohérent. L'impédance de sortie est égale à l'impédance d'entrée !

Mardi Après-midi : Etude du banc de test Analogique



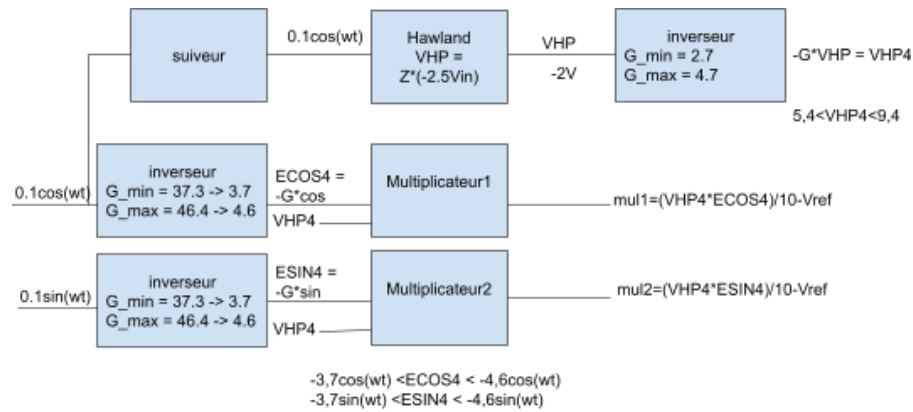
Nous pouvons décomposer notre schéma électronique en plusieurs blocs dont :

- 3 AOP Inverseur TL082P (rouge)
- 1 AOP suiveur TL082P (rose)
- 1 source de courant de Howland LM675T (violet)
- 2 multiplieurs analogiques MPY634KP (vert)

Sur ce deuxième schéma nous avons l'implantation exacte de chaque composant et nous avons dû faire la correspondance avec le premier schéma pour bien comprendre comment nous allons faire les tests de la carte analogique.

- Nous retrouvons les 2 multiplieurs , 2 AOP inverseurs et la source de Howland avec leur potentiomètres respectifs pour régler leurs gains , nous avons aussi un potentiomètre qui nous permet de faire varier la tension Vref entre 0 et 2.5V

- N'oublions pas de remarquer aussi l'espace à droite de notre carte ou nous viendrons connecter le filtre passe-bas que nos collaborateurs ont conceptionné pour pouvoir récupérer des signaux épurés pour pouvoir les analyser et calculer l'impédance de notre haut parleur



Par la suite nous avons pu élaborer ce schéma fonctionnel qui simplifie et condense le fonctionnement de notre système , les gains minimums et maximums y sont inscrits pour pouvoir savoir à l'avance quels types de résultats nous devons attendre et quels réglages nous devons faire pour que le banc de test fonctionne de manière optimale et fiable

Mercredi matin : Etude du CAN et de la FFT pour passage en numérique

```
void loop() {  
  float vdiff;  
  int value = read_adc();  
  
  // if MSB = 1 negative output code  
  if (value & 0b1000000000) { value |= 0b1111111111111111111100000000; }  
  
  vdiff = (value * vref / 512.0);  
  
  Serial.print(value,BIN);  
  Serial.print(" ");  
  Serial.print(vdiff);  
  Serial.print("\r\n");  
  
  _delay_ms(2000);  
}
```

Ce programme arduino nous permet de récupérer un signal analogique de le convertir et de l'échantillonner afin de récupérer au final un signal numérique

- Pour ce faire nous avons utilisé un générateur de fonctions qui nous a permis d'injecter un signal sinusoïdal (idéalement positif pour ne pas endommager notre arduino mega) dans notre micro-contrôleur
- Notre micro-contrôleur ne possédant pas d'entrées différentielles nous devons utiliser deux entrées analogiques , une comme entrée positive et l'autre comme entrée négative
- Nous avons ensuite observé sur le moniteur série les valeurs converties

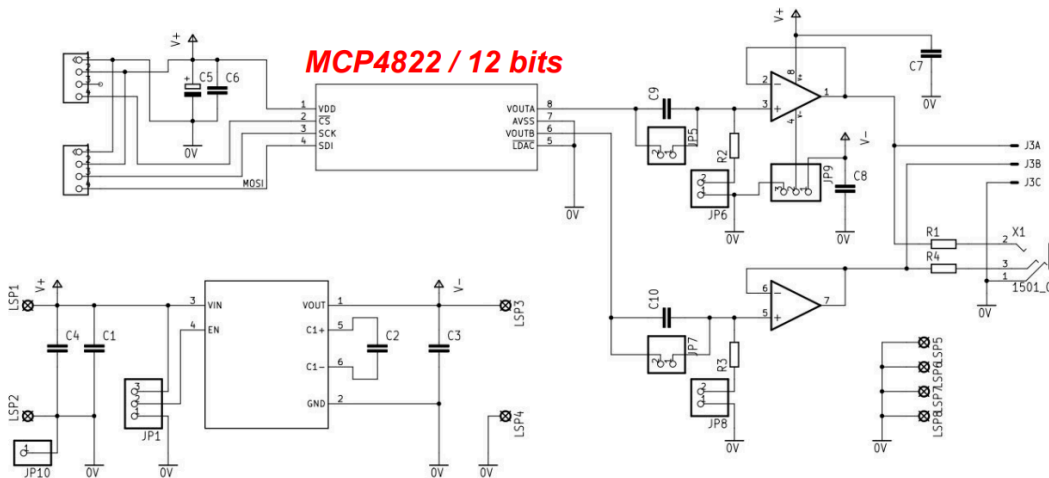
```
for (int i = 0; i < 64; i++) {  
  data[i]=read_adc();  
  delay(1);  
}  
  
FFT(data, 64, 200);
```

Ce programme arduino quant à lui nous permet d'effectuer la FFT du signal que nous avons acquis grâce au programme précédent

- Pour effectuer la FFT nous devons spécifier le nombre d'échantillons (ici 64) et la fréquence d'échantillonnage (ici 200Hz (100*2) pour respecter le théorème de Shannon)
- Notre objectif avec programme est d'afficher la valeur moyenne du signal converti par le CAN , pour calculer la valeur moyenne nous utilisons l'amplitude de la composante continue que nous multiplions par le quantum et que nous divisons ensuite par le nombre d'échantillons
- Avec un signal d'amplitude 5Vpp et d'offset 2.5V nous avons récupéré une valeur moyenne de 2.5V ce qui est tout à fait cohérent

```
rial.println((out_r[0]*vref/512)/64);
```

Mercredi après-midi : Génération des signaux par SPI avec le CNA



Ce schéma électronique correspond à la carte qui contient le MCP4822 le CNA qui va nous permettre en communiquant en SPI avec notre arduino , de retranscrire les signaux numériques en signaux analogiques que l'on va pouvoir traiter pour notre étude

- le MCP4822 est un convertisseur numérique-analogique de 12 bits
- Nous allons devoir communiquer en SPI avec notre arduino Mega , le CNA étant l'esclave et l'arduino le maître

MCP4822 Pin	Arduino Mega Pin	C'est grâce à ce tableau que nous allons pouvoir relier et effectuer le cable entre le micro-contrôleur et le CNA - CS sert à choisir l'esclave avec lequel l'arduino veut communiquer - SCK sert à synchroniser le maître et l'esclave avec un signal d'horloge - SDI sert au transfert de données entre les différents appareils
VDD	5V	
VSS	GND	
CS	Pin 53	
SCK	Pin 52	
SDI	Pin 51	

```

#include <SPI.h> // Inclure la bibliothèque SPI
#define CS_PIN 53 // Broche Chip Select (SS) pour le DAC
#define N 100 // Nombre de points dans la sinusoïde/cosinus
#define AMPLITUDE 2000 // Amplitude de la sinusoïde/cosinus (pour un DAC 12 bits)
#define OFFSET 2000 // Décalage pour obtenir des valeurs positives
#define DAC_MAX 4095 // Valeur max pour un DAC 12 bits

uint16_t sinTable[N]; // Tableau pour stocker les points de la sinusoïde
uint16_t cosTable[N]; // Tableau pour stocker les points du cosinus

void setup() {
    pinMode(CS_PIN, OUTPUT); // Configurer la broche CS comme sortie
    SPI.begin(); // Initialiser la communication SPI

    // Générer les points de la sinusoïde et du cosinus
    generateSinTable();
    generateCosTable();
}

void loop() {
    // Envoyer les points du sinus et du cosinus continuellement
    for (int i = 0; i < N; i++) {
        sendToDAC(sinTable[i], 0); // Envoyer au DAC A (canal 0 - sinusoïde)
        sendToDAC(cosTable[i], 1); // Envoyer au DAC B (canal 1 - cosinus)
        delayMicroseconds(73); // Ajuster pour contrôler la fréquence (100 Hz)
    }
}

// Générer les points de la sinusoïde
void generateSinTable() {
    for (int i = 0; i < N; i++) {
        sinTable[i] = (uint16_t)(AMPLITUDE/11 * sin(2 * M_PI * i / N) + OFFSET);
    }
}

// Générer les points du cosinus
void generateCosTable() {
    for (int i = 0; i < N; i++) {
        cosTable[i] = (uint16_t)(AMPLITUDE/11 * cos(2 * M_PI * i / N) + OFFSET);
    }
}

// Envoyer des données au MCP4822 via SPI
void sendToDAC(uint16_t value, uint8_t channel) {
    // Configurer la commande: bit 15 = 0 pour DAC A, 1 pour DAC B
    uint16_t command = (channel << 15) | (0 << 13) | 0x3000 | (value & 0x0FFF);

    digitalWrite(CS_PIN, LOW); // Activer le DAC (CS bas)
    SPI.transfer16(command); // Envoyer les 16 bits via SPI
    digitalWrite(CS_PIN, HIGH); // Désactiver le DAC (CS haut)
}

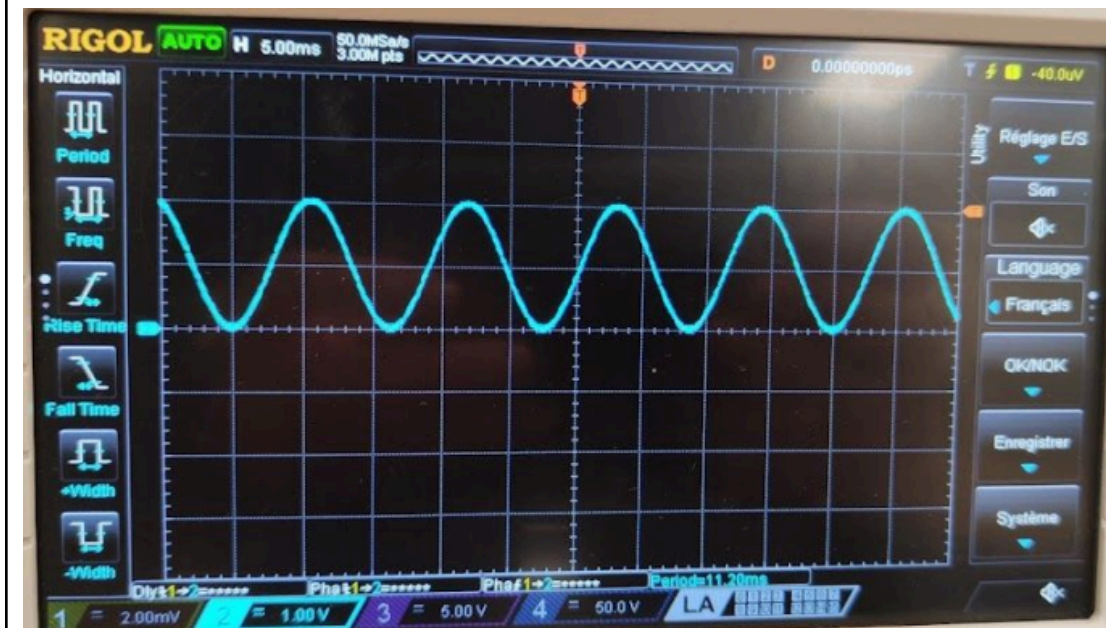
```

Voici maintenant le programme élaboré pour ne plus utiliser de GBF et pouvoir générer nos signaux directement numériques avec notre micro-contrôleur, le sinus et cosinus sont créés dans un tableau (ici l'amplitude est divisée par 11 pour pouvoir limiter les signaux à 200 mVpp)

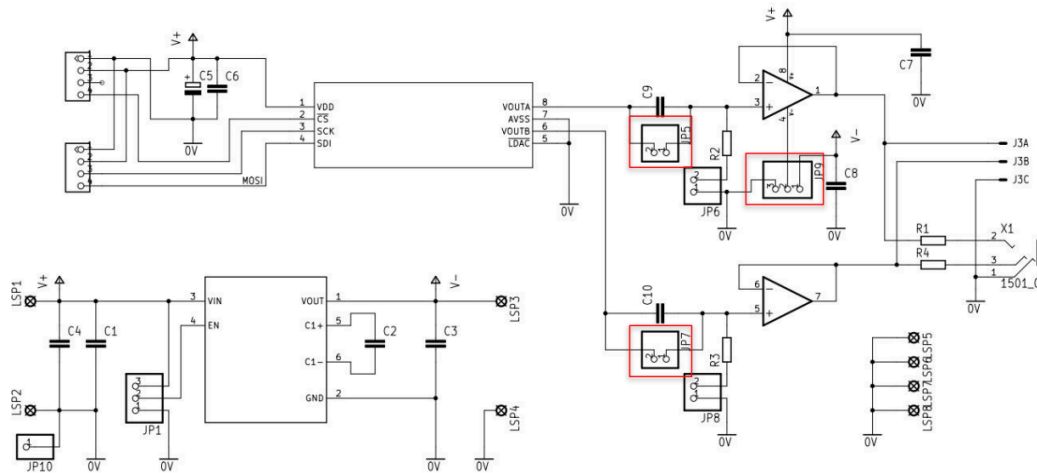
- Chaque période de notre sinus/cosinus est composée de 100 points et la fréquence de notre signal a une fréquence de 100Hz

- Normalement pour avoir cette fréquence de 100Hz nous aurions dû mettre un délai de 100 microSecondes entre chaque envoi de données mais nous avons remarqué que l'envoi de données par SPI prend un certain temps c'est pourquoi nous avons dû ajuster notre délai pour atteindre la fréquence voulue.

- Le seul problème étant que les signaux générés avaient un offset et donc n'avaient pas de parties négatives, nous devons régler ce problème le lendemain matin



Jeudi : Réglage du problème d'offset de nos signaux + mise en commun du CNA avec la source de courant Howland + essai de FFT et mise en place du double CAN



Après une étude approfondie de la carte où se trouve notre MCP4822, nous avons remarqué qu'il y'a des jumpers pour changer le comportement de notre circuit

- Nous avons d'abord modifié la position du jumper marqué '0/-5V' afin de permettre le traitement des valeurs négatives
- Ensuite nous avons modifié la position des jumpers : au lieu de court-circuiter les condensateurs, les jumpers sont désormais positionnés pour court-circuiter les résistances
- Grâce à ces réglages, nous avons pu obtenir des signaux purement sinusoïdaux en sortie de cette carte



Pour la suite de la numérisation de notre système nous devons ajouter le filtre de Howland, pour ce faire nous allons utiliser la carte analogique que nous avons étudié, mais sans utiliser les multiplieurs et les autres composants

- Nous avons élaboré un plan d'action pour pouvoir atteindre notre objectif final qui est de calculer l'impédance de notre haut parleur simulé par une résistance de 8 Ohms

1. générer un cosinus et l'injecter dans le Howland
2. ensuite on récupère la tension aux bornes du Haut-Parleur soit après Howland
3. faire la corrélation sinusoïdale sur arduino
4. faire la FFT avec le résultat de la multiplication
5. faire les calculs pour retrouver l'impédance du haut parleur

```

void loop() {
  // Lire et afficher les résultats du CAN 1 (ADC0, ADC1)
  float vdiff_CAN1;
  int value_CAN1 = read_adc(CAN1_input);
  vdiff_CAN1 = process_adc_value(value_CAN1);

  Serial.print("CAN 1 (ADC0-ADC1): ");
  Serial.print(value_CAN1, BIN); // Afficher la valeur binaire brute
  Serial.print(" ");
  Serial.print(vdiff_CAN1); // Afficher la tension en volts
  Serial.print("V");
  Serial.print("\r\n");

  // Lire et afficher les résultats du CAN 2 (ADC2, ADC3)
  float vdiff_CAN2;
  int value_CAN2 = read_adc(CAN2_input);
  vdiff_CAN2 = process_adc_value(value_CAN2);

  Serial.print("CAN 2 (ADC2-ADC3): ");
  Serial.print(value_CAN2, BIN); // Afficher la valeur binaire brute
  Serial.print(" ");
  Serial.print(vdiff_CAN2); // Afficher la tension en volts
  Serial.print("V");
  Serial.print("\r\n");

  _delay_ms(2000); // Pause de 2 secondes entre chaque cycle de mesure
}

```

Grâce à ce programme nous effectuons 2 conversions analogiques-numériques qui vont nous servir à effectuer la corrélation sinusoïdale et par la suite de faire la FFT pour enfin calculer l'impédance de notre haut-parleur

- La première conversion sert à acquérir le signal que nous injectons dans le Howland
- La deuxième conversion nous sert à acquérir le signal récupéré après le Howland soit aux bornes du Haut-Parleur
- L'inconvénient de ce mode opératoire est que les 2 conversions ne font pas simultanément, il faudra en prendre compte lors de la multiplication de nos signaux en compensant le/les échantillons perdus lors du délai entre les 2 conversions ou bien générer le sinus d'origine directement dans le programme qui effectue la FFT

Conclusion

Ce projet a permis de poser les bases d'un banc de mesure fiable pour l'analyse de l'impédance des haut-parleurs, intégrant des techniques analogiques et numériques pour une étude approfondie. L'objectif final – mesurer précisément l'impédance de notre haut-parleur – n'a pas pu être entièrement atteint en raison de contraintes de temps. Cependant, les résultats obtenus montrent que cet objectif reste réalisable avec un approfondissement des programmes Arduino et une optimisation de la chaîne d'acquisition numérique.

Pour mener ce projet, nous avons mobilisé un ensemble de compétences variées. L'électronique analogique a été mise en œuvre à travers la source de courant de Howland et le multiplicateur MPY634KP. Côté numérique, le projet a intégré une chaîne d'acquisition complète, combinant un convertisseur analogique-numérique (CAN) pour la capture des signaux et un convertisseur numérique-analogique (CNA) pour générer des signaux de test précis, ainsi que des techniques de traitement du signal en fréquence.

L'implémentation de la FFT dans cette chaîne d'acquisition numérique s'est avérée essentielle pour les analyses fréquentielles en temps réel. Ce système pourrait encore être amélioré avec une optimisation des programmes Arduino, renforçant ainsi la précision des mesures et permettant de détecter les plus petites anomalies des haut-parleurs.

En conclusion, ce projet pose des bases solides pour une automatisation complète et pour des mesures encore plus précises avec un système numérique amélioré. Les résultats offrent des outils et des méthodes robustes pour des études futures, ouvrant la voie à une compréhension approfondie des caractéristiques d'impédance des haut-parleurs.